

경력 기술서

전소현 | Frontend Developer

React·TypeScript 기반 실시간 관제 / 데이터 시각화 프론트엔드 개발자

2025.05 ~ 현재 | 시티아이랩(주) 기업부설연구소

실시간 영상·센서 관제 시스템과 디지털 트윈 기반 데이터 시각화 서비스를 개발해 왔습니다. UI/UX 설계 및 구현, WebSocket 기반 실시간 데이터 처리, 영상 스트리밍, Canvas·3D 시각화, Electron 현장 배포까지 프론트엔드 영역 전반을 담당했습니다.

핵심 성과

- 24시간 미만 주기로 발생하던 **Electron 화이트 스크린과 메모리 이상향 문제**를 JS Heap → renderer memory → ECharts 렌더링 경로까지 단계적으로 분석해 **해소**
- 고해상도 지도 **Painting Time 4,300ms → 170ms(96%), Main Thread Long Task 51회 → 7회(86%)**로 개선
- 수 GB 규모 FCD 데이터 처리 구조를 온디맨드 스트리밍 방식으로 재설계해 **처리 시간 400s → 85s(78.7%)**로 단축하고 Unity WebGL OOM 해소
- 9개 도메인의 API 구조를 TanStack Query 기반으로 재설계해 메인 페이지 **API 관련 코드 약 500줄 감축**
- JavaScript → TypeScript 전환 및 도메인 타입 구축으로 타입 관련 **런타임 오류 4건 → 0건 감소**

전기차 화재 감지 시스템

2025.05 - 현재

Frontend Developer

- **적용/배포:** 대기업, 공공기관 및 공공시설 현장
- **담당:** UI/UX 설계 및 프론트엔드 개발, 영상 스트리밍·통합 배포 구조 설계, 백엔드 API 유지보수, Electron 통합 배포
- **팀 구성:** Frontend 1명, Backend 3명

프로젝트 개요

- 가시광·열화상 카메라와 이기종 센서를 통합한 실시간 화재 감지·관제 시스템
- 영상·온도 데이터를 기반으로 ROI 관리, 이상 온도 알람
- AI 영상 분석 결과를 통해 화재·연기·사람 탐지 결과 시각화

기술 스택

- **Frontend:** React · TypeScript · TanStack Query
- **Visualization:** Canvas API · ECharts · Three.js
- **Realtime:** WebSocket · REST API

-
- **Media:** RTSP · HLS · MediaMTX · FFmpeg
 - **Desktop / Deployment:** Electron · PyInstaller
 - **Collaboration:** GitLab · Git LFS · Jira

주요 구현

- 별도 운영되던 AI 영상 기반 천장형 관제와 열화상·센서 기반 바닥형 관제를 단일 프론트엔드 플랫폼으로 통합
- Canvas 기반 ROI 편집·BBox 실시간 렌더링 구현
- MediaMTX·FFmpeg 기반 RTSP → HLS 스트리밍 구축
- QGIS·Three.js 기반 3D 공간 관제 구현
- Electron 기반 현장 배포 구조 구축

1. 화이트 스크린 및 메모리 증가 문제 해결

a. Problem

- 24시간 상시 가동되는 현장 환경에서 일정 시간 주기로 화면이 하얗게 멈추는 장애 발생
- 실행 시간이 길어질수록 메모리가 지속적으로 증가해 장기 관제가 어려움
- 단순 React 상태나 JavaScript 객체만 점검해서는 원인이 명확하게 확인되지 않음

b. Analyze

- Chrome DevTools의 **Heap Snapshot · Allocation Timeline**으로 JavaScript 객체의 생성·해제 추이와 Retainer 경로를 분석하고, 비동기 작업 및 외부 리소스의 생명주기를 점검
- JavaScript Heap 관련 누수 요소를 정리한 이후에도 **프로세스 메모리가 지속적으로 증가하는 현상을 확인**해 JS Heap 외부로 분석 범위 확대
- Windows 작업 관리자의 PID별 활성 개인 메모리 · 커밋 크기를 추적하며 **Electron renderer memory 증가 문제로 범위를 좁힘**
- 컴포넌트와 렌더링 기능을 단계적으로 비활성화해 비교한 결과, **ECharts의 반복적인 setOption() 실행 경로에서 renderer memory가 증가하는 현상 확인**
- axis · series · animation을 각각 분리해 검증하며 동적 X축 시간 **formatter**의 반복 렌더링 경로까지 원인을 세분화

c. Action

i. 리소스 생명주기 및 실시간 처리 구조 개선

- 비동기 요청 취소, WebSocket·Worker·Timer·RAF·Observer 등 외부 리소스 cleanup을 정비하고 ImageBitmap.close()를 적용해 컴포넌트 종료 이후 남은 작업과 네이티브 이미지 리소스를 명시적으로 해제
- 실시간 데이터에 고정 크기 **rolling window**를 적용해 무제한 누적을 방지
- 열화상 프레임 처리에 최신 프레임 우선 **backpressure**를 적용해 이전 비동기 디코딩 작업의 중첩을 방지

ii. ECharts 렌더링 구조 개선

- 실제 시각이 계속 변경되던 X축을 고정된 상대 좌표 범위로 전환하고 데이터 좌표만 최신 시각 기준으로 변환해 축 모델의 반복 갱신 방지

-
- 실제 시간 라벨을 ECharts Canvas에서 **React DOM으로 분리**하고 고정 DOM 노드 재사용
 - 시간 라벨 갱신 주기를 초 단위로 제한하고 메모이제이션 적용

iii. 장애 복구 구조 구축

- WebSocket 재연결을 고정 주기 방식에서 **최대 대기 시간을 둔 지수 백오프** 방식으로 변경
- React · Electron 오류를 통합 수집하고 IPC 기반으로 백엔드에 전달하는 오류 수집 구조 구축
- 렌더링 오류 발생 시 fallback UI 제공 후 지연 새로고침 수행
- renderer 종료나 응답 없음 상태를 Electron main process에서 감지해 단계적 reload 및 BrowserWindow 재생성을 수행하는 자동 복구 절차 구축
- localStorage 기반 상태 영속화를 적용하여 renderer 복구 후 기존 관제 상태 복원

f. Result

- **24시간 미만 주기로 발생하던 화이트 스크린 현상 해소**
- **JavaScript Heap 및 Electron renderer memory의 지속적인 이상향 현상 안정화**
- renderer · WebSocket · 백엔드 장애 발생 시 자동 복구 및 관제 상태 복원 구조 확보
- 프론트엔드 오류를 백엔드로 수집하는 체계를 구축해 장시간 운영 환경의 장애 진단 범위 확대

2. JavaScript → TypeScript 전환 및 컴포넌트 구조 개선

a. Problem

- 천장형·바닥형 관제 시스템 통합 과정에서 기능과 데이터 흐름이 복잡해지며 **JavaScript의 타입 부재와 책임이 혼재된 컴포넌트 구조**로 인한 유지보수 문제 발생
- 기능 변경 시 관련 로직이 여러 컴포넌트에 분산되어 **변경 영향 범위와 오류 발생 지점 추적이 어려움**
- API · WebSocket 데이터의 타입이 보장되지 않아 예상하지 못한 런타임 오류 발생
- 실시간 데이터를 하나의 WebSocket Context에서 관리해 특정 데이터 변경 시 **해당 데이터를 사용하지 않는 컴포넌트까지 함께 리렌더링**

b. Analyze

- 시스템 통합 이후에도 동일 구조에서 신규 기능을 계속 추가할 경우 **런타임 오류와 컴포넌트 간 결합도가 함께 증가할 것으로 판단**
- 단순 코드 수정이 아니라 **타입 안정성 확보와 컴포넌트·실시간 상태의 책임 분리**가 선행되어야 한다고 판단해 신규 기능 개발보다 구조 개선을 우선

c. Action

- JavaScript 코드를 TypeScript로 전환해 컴파일 단계에서 타입 오류 검증
- 핵심 비즈니스 도메인의 타입을 정의해 API · WebSocket 데이터와 컴포넌트 사이의 데이터 계약을 명시
- 단일 책임 원칙을 기준으로 컴포넌트 역할과 폴더 구조 재설계
- 단일 WebSocket Context를 역할별 Context로 분리하고, 각 컴포넌트가 필요한 상태만 구독하도록 데이터 흐름 개선

d. Result

- TypeScript 전환 이후, **타입 관련 런타임 오류 4건 → 0건**
- 컴포넌트와 도메인별 책임을 분리해 기능 변경 시 확인해야 하는 코드 범위와 의존 관계를 명

확하게 개선

- WebSocket 상태 변경 시 전체 Context 소비자가 함께 갱신되던 구조를 분리해 **실시간 데이터 변경에 따른 리렌더링 전파 범위 축소**

3. API 아키텍처 재설계 및 TanStack Query 도입

a. Problem

- 메인 페이지에 **9개 도메인의 API 호출과 서버 상태 관리 로직이 집중**되어 컴포넌트 책임 증가
- 각 API에서 AbortController, timeout, try-catch, snake_case → camelCase 변환 로직이 반복
- 서버 응답 변경 여부를 확인하기 위해 JSON.stringify(prev) === JSON.stringify(next) 기반 깊은 비교 반복
- fetch 함수와 상태 setter가 prop drilling으로 전달되며 컴포넌트 간 결합도 증가
- 동일 서버 데이터를 여러 컴포넌트에서 개별 호출해 중복 요청 및 상태 불일치 가능성 존재
- API마다 에러 응답 처리 방식이 달라 예외 처리의 일관성 부족

b. Analyze

- 개별 fetch 코드를 정리하는 것만으로는 **중복 요청 · 상태 동기화 · 에러 처리 · 데이터 변환 문제를 함께 해결하기 어렵다**고 판단
- UI 상태와 서버 상태가 컴포넌트 내부에서 함께 관리되는 구조를 분리하고, **HTTP 통신 → API 응답 변환 → 서버 상태 관리의 책임을 계층별로 분리하기로 결정**
- polling · 캐싱 · mutation 이후 재조회가 반복되는 특성을 고려해 직접 상태 동기화 대신 **Query Cache 기반 서버 상태 관리 구조가 적합하다**고 판단

c. Action

i. 공통 HTTP 계층 구축

- httpClient에서 timeout · 응답과 parsing · 공통에러 처리 로직을 통합해 API 호출 보일러 플레이트 제거
- HttpError 커스텀 에러를 통해 status와 response body 기반 예외 처리 방식 통일

ii. API 레이어 분리

types/{domain}.api.ts → services → hooks/queries

- 서버의 raw response 타입과 프론트엔드 domain model을 분리해 컴포넌트가 서버 응답 형식에 직접 의존하지 않도록 구성
- HTTP 호출과 snake_case → camelCase 변환을 service 계층에 캡슐화
- query hook으로 데이터 fetching과 서버 상태 관리 책임을 컴포넌트 외부로 분리

iii. TanStack Query 도입

- queryKey 기반 캐시를 적용해 동일 서버 데이터를 여러 컴포넌트에서 공유
- polling 및 request deduplication을 Query 계층으로 이동
- mutation 이후 invalidateQueries를 통해 관련 서버 상태를 재동기화
- structural sharing을 활용해 변경되지 않은 데이터 참조를 유지하고, 깊은 비교 로직 제거

c. Result

- 메인 페이지 기준, API 관련 코드 약 500줄 감축
- 컴포넌트 내부의 직접 fetch 및 반복적인 API 보일러플레이트 제거

-
- 서버 상태를 Query Cache에서 공유하도록 변경해 **fetch 함수 · 상태 setter의 prop drilling 제거**
 - 동일 query를 여러 컴포넌트가 사용하는 경우 개별 fetch하던 구조를 제거
 - JSON.stringify 기반 **직접 상태를 비교하고 동기화하는 로직 제거**
 - API 응답 타입·데이터 변환·에러 처리 방식을 도메인별 동일한 구조로 표준화
-

교통 신호 운영 분석 소프트웨어

2025년 07월 - 2026년 01월

Frontend Developer

- **적용/배포:** 내부 PoC 및 상용화를 위한 영업 단계
- **담당:** UI/UX 설계, 프론트엔드·Unity 개발, FCD 데이터 파이프라인 구현
- **팀 구성:** Frontend 1명, Backend 2명

프로젝트 개요

- SUMO · Unity · React를 연동한 디지털 트윈 기반 실시간 교통 시뮬레이션 시스템
- 교차로 신호 최적화 전 · 후의 교통 흐름을 3D 시뮬레이션하고 성능 비교 · 통계 시각화 제공

기술 스택

- **Frontend:** React · TypeScript · Redux Toolkit · Canvas API · Recharts
- **3D / Simulation:** Unity WebGL · C# · QGIS · glTF
- **Data Processing:** Python · lxml · NDJSON
- **Realtime:** WebSocket · REST API
- **Deployment:** Electron

주요 구현

- React에 Unity WebGL을 임베드해 3D 교통 시뮬레이션 구현
- 교차로 선택 → 모델 생성 → 최적화 → 이력 → 성능 비교 대시보드 구현
- Unity C# WebSocket 구조를 구축해 FCD 데이터를 시각화 모듈에 전달
- SUMO 차량 좌표를 Unity 공간에 정합하는 좌표 변환 로직 구현
- QGIS 기반 지형·건물 데이터를 가공해 Unity 환경에 적용

1. 고해상도 지도 Canvas 렌더링 병목 해결

a. Problem

- 마우스 드래그 · 줌 과정에서 **고해상도 지도와 다수 노드를 동시에 렌더링하며 심각한 프레임 저하 발생**
- 줌아웃 시 고해상도 원본 이미지를 매 프레임 축소하면서 **반복적인 이미지 리샘플링 및 Painting 비용 증가**
- 지도와 인터랙션 요소를 동일한 Canvas에서 반복 렌더링해 정적 영역까지 불필요하게 다시 그리는 구조 확인

b. Analyze

- Chrome DevTools Performance로 렌더링 구간을 분석한 결과, JavaScript 실행 시간보다 **고해상도 이미지의 반복적인 Painting 비용이 주요 병목임을 확인**
- 원본 이미지를 매 프레임 축소하는 비용을 계속 지불하는 대신 **추가 메모리를 사용해 축소 이미지를 사전 생성·캐싱하는 방식이 인터랙션 성능에 유리하다고 판단**
- 정적 지도와 동적 인터랙션 요소의 갱신 주기가 다름에도 동일 Canvas에서 렌더링되는 구조가 불필요한 재그리기를 발생시킨다고 판단

c. Action

- 지도 이미지를 ImageBitmap으로 디코딩해 useRef에 캐싱하고, 언마운트 시 .close()를 호출해 이미지 리소스를 명시적으로 해제
- **정적 지도와 동적 인터랙션 레이어를 분리한 Multi-Canvas 구조를 적용해** 갱신이 필요한 영역만 다시 렌더링하도록 개선
- 50%·25% 축소 이미지를 오프스크린 Canvas에 사전 생성하고 줌 배율에 따라 적절한 해상도의 이미지를 선택하는 **맵매핑 방식**을 설계
- PNG를 WebP로 변환하고 quality=75 손실 압축을 적용해 초기 이미지 로딩 비용 축소
- WebP 변환 과정에서 해상도가 자동 조정되며 기존 노드 좌표와 불일치하는 문제를 발견하고, **원본 대비 해상도 비율을 기준으로 좌표와 offset을 동적으로 보정**

d. Result

Chrome DevTools Performance 기준:

- **지도 로딩 시간 약 10s → 3s**
- **Painting Time 약 4,300ms → 170ms, 96% 감소**
- **Main Thread Long Task 51회 → 7회, 86% 감소**

2. 대용량 FCD 처리 최적화 및 전체 적재 OOM 개선

a. Problem

- SUMO에서 생성되는 FCD XML 데이터가 수 GB 규모로 증가하면서 **전체 변환·적재 방식에서 약 400초의 처리 시간 발생**
- 변환된 전체 데이터를 Unity WebGL에 적재할 경우 브라우저 메모리 한계로 **OOM 발생**
- 대용량 시나리오에서 전체 데이터를 사전에 변환하고 메모리에 보관하는 구조의 확장성 한계 확인

b. Analyze

- 1차로 XML → JSON 변환과 필요 필드 선별을 통해 데이터 크기를 줄였으나 **처리 시간은 개선됐지만 Unity WebGL OOM은 지속**
- 단순 포맷 경량화로는 브라우저의 메모리 한계를 해결하기 어렵다고 판단하고, **전체 데이터를 보유하는 방식 자체를 제거하는 방향으로 문제를 재정의**
- 시뮬레이션에서 특정 시점의 데이터만 필요하다는 특성을 기반으로 **전체 적재 대신 필요한 구간만 조회·전송하는 온디맨드 스트리밍 구조를 설계**

c. Action

- XML 전체 변환·적재 방식에서 **NDJSON 기반 스트리밍 구조로 전환**

-
- 필요한 시뮬레이션 시점의 데이터만 WebSocket으로 전달하는 온디맨드 데이터 공급 구조 구현
 - 시뮬레이션 시간과 파일 내부 byte offset을 매핑한 인덱스 테이블을 사전에 구축
 - 요청 시 파일 전체를 순차 탐색하지 않고 **인덱스에 기록된 byte offset으로 직접 찾아 필요한 데이터 구간을 조회하도록 개선**
 - Python으로 데이터 처리 파이프라인을 구현하고 Java 백엔드에서 활용할 수 있도록 구조와 처리 방식을 인계

d. Result

- **FCD 데이터 처리 시간 약 400s → 85s, 약 78.7% 단축**
 - 포맷 및 필드 경량화: 약 400s → 115s
 - 인덱스 기반 조회 구조 적용: 115s → 85s
 - **FCD 전체 데이터를 Unity WebGL 메모리에 적재하면서 발생하던 OOM 현상 제거**
 - 전체 적재 방식에서 필요한 시점의 데이터만 공급하는 구조로 전환해 **FCD 데이터 규모 증가에 따른 클라이언트 메모리 부담 완화**
-